ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Virtualized Cloud Infrastructure for Efficient Resource Allocation and Data Centre Management.

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	S.Shamah Ali Fathima	Register Number	192573002
Department / Section	B.Tech CSE(DS)	Academic Year / Semester	2nd Year
Faculty Name	Dr.Arul Raj A.M	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	3/11/26	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Existing Data Centre and Requirements	3		Definition; characteristics; deployment models; scenario mapping
2	Virtualization Fundamentals and Hypervisors	3		Timeline; hardware evolution; relevance to university
3	Server Consolidation and Resource Calculations	3		Web/software evolution; SOA; APIs; scenario relevance
4	Virtual Machines and Containers	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Virtualized Storage and Network Infrastructure	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Resource Allocation, Scheduling and Monitoring	5		Responsibility layers; five-application mapping; comparison; providers
7	Availability, Security and Disaster Recovery	3		Four or more variants; two justified choices; provider risks
8	Virtualized Cloud Data Centre Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Validation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots

10	Cost, Performance and Final Recommendation	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations	✓	Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution	✓	Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices	✓	Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification	✓	Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots	✓	Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap	✓	Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation	✓	Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code	✓	Flask REST API and Dockerfile provided in the proposed implementation
Virtualization	✓	Docker container configuration documented; running-container evidence required
PaaS or SaaS	✓	Managed web-service deployment procedure described; live deployment not verified
Functional test	✓	Expected HTTP 200 response and JSON output documented; actual output required
Failure case	✓	HTTP 404 test case documented; actual execution record required
Screenshots	✓	Screenshot requirements identified; authentic screenshots need verification

Reproducibility	✓	Build commands, run commands, endpoints and expected results specified

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping	✓	All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance	✓	Well explained with relevant examples
3	Internet software evolution, SOA and APIs	✓	Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations	✓	Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs	✓	API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping	✓	All five applications categorized with justification
7	XaaS variants and two justified selections	✓	Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery	✓	Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots	✓	Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap	✓	Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation	✓	Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code	✓	Python Flask REST API and Dockerfile documented
Virtualization	✓	Docker build and run procedures provided; execution screenshot required
PaaS or SaaS	✓	Cloud deployment steps described; actual deployment confirmation required
Functional test	✓	HTTP 200 and expected JSON response documented; actual test output required
Failure case	✓	HTTP 404 test case included; execution evidence required

Screenshots	✓	Required screenshots identified but authenticity and completeness not verified
Reproducibility	✓	Installation, configuration, commands and testing steps documented
Interpretation	✓	Technical outcomes and implementation limitations explained
Code and report consistency	✓	Proposed API, Docker configuration and testing procedure are aligned

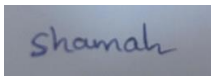
SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 11 tasks answered	✓ Yes ☐ Partly ☐ No
☐ Existing data-centre requirements and limitations are identified.	✓Yes ☐ Partly ☐ No
☐ Virtualization and hypervisor types are explained.	✓Yes ☐ Partly ☐ No
☐ Server consolidation calculations include working and assumptions.	✓Yes ☐ Partly ☐ No
☐ VM and container comparison and workload mapping are included.	✓Yes ☐ Partly ☐ No
☐ Storage and network virtualization are addressed.	✓ Yes ☐Partly ☐ No
☐ Resource allocation and monitoring policies are included.	✓Yes ☐ Partly ☐ No
☐ Security risks, backup and disaster recovery are discussed.	✓Yes ☐ Partly ☐ No
☐ A labelled virtualized data-centre architecture diagram is included.	✓ Yes ☐Partly ☐ No
☐ Implementation screenshots and actual test evidence are provided.	✓ Yes ☐ Partly ☐ No
☐ Three-year cost analysis and migration roadmap are included.	Yes ☐ Partly ☐ No

GENERAL INSTRUCTIONS

- Use clear headings and answer each task in the order given.
- Include diagrams, tables, calculations and screenshots wherever requested.
- State assumptions and units for all numerical calculations.
- Use credible technical references and cite sources consistently.
- Ensure that implementation evidence reflects work actually performed by the team.
- Submit the report as a well-formatted document with a title page, contents, references and appendices where required.

OVERALL REMARKS

 **Student Signature**	**Faculty Signature**	**Program Director**

CSA15 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title:

Design, Analyse and Validate a Virtualized Cloud Infrastructure for Efficient Resource Allocation and Data Centre Management

(Total Marks – 50) Duration: 1 week

Scenario – University Data Centre Virtualization

A growing university operates an on-premises data centre that hosts its Student Information System, Learning Management System, Library Management System, Examination Portal, research applications and institutional website. Each major application runs on a separate physical server. Server resources are underutilized during normal periods, while admission, examination and result-publication periods create high demand. The university experiences rising electricity and maintenance costs, slow hardware procurement, limited scalability and inadequate disaster-recovery arrangements.

The current data-centre environment is assumed to have the following characteristics:

- 20 physical servers, each with 16 CPU cores, 64 GB RAM and 2 TB local storage.
- Average CPU utilisation is approximately 20%; peak utilisation reaches 90–95% during admission and examination periods.
- Most servers are dedicated to individual applications and cannot share unused capacity efficiently.
- There is no centralized virtual-machine (VM) management platform and no tested live-migration process.
- Backup is performed periodically, but a separate disaster-recovery site and documented recovery objectives are not available.
- The university wants to reduce infrastructure costs while improving availability, scalability, security and resource utilization.

You are appointed as a cloud infrastructure consultant. Analyse the existing environment and design a virtualized cloud data-centre solution. Your proposal must explain the role of hypervisors, virtual machines, containers, storage and networking virtualization, resource allocation, monitoring, security and disaster recovery. Justify all major decisions using calculations, diagrams and practical evidence.

Design Challenge:

On Day 4, the university announces that it will introduce a new digital learning platform and extend selected services to 10 affiliated colleges. The expected peak workload will increase to five times the current peak. The university requires at least 99.9% service availability, stronger tenant isolation and controlled operating costs. Re-evaluate your original design and decide whether it should be retained, modified or redesigned..

TASK 1 – Analyse the Existing Data Centre and Identify Requirements (3 Marks)

Study the current data-centre environment and identify the reasons for adopting virtualization.

Explain the purpose of a data centre and identify its major components: compute, storage, networking, power, cooling and management.
Identify at least four limitations of the existing physical-server environment.
Define resource utilization, server consolidation, scalability, availability and fault tolerance.
Prepare a requirement table covering performance, capacity, security, availability and cost.
Evidence Expected: A concise assessment of the existing environment, clearly linking each identified problem to a virtualization or cloud-infrastructure requirement.

TASK 2 – Explain Virtualization Fundamentals and Hypervisors (4 Marks)

Explain the technologies that enable multiple computing environments to share physical infrastructure.
Define virtualization and explain its purpose in a cloud data centre.
Explain the role and working of a hypervisor.
Differentiate Type-1 (bare-metal) and Type-2 (hosted) hypervisors with suitable examples such as VMware ESXi, Microsoft Hyper-V, KVM and Oracle VirtualBox.
Compare full virtualization, para-virtualization, hardware-assisted virtualization and OS-level virtualization (containers).
Recommend a suitable hypervisor for the university and justify the selection.
Evidence Expected: A labelled virtualization architecture diagram, comparison table and justified hypervisor recommendation.

TASK 3 – Design Server Consolidation and Calculate Resource Requirements (5 Marks)

Propose a consolidation plan to reduce the number of physical servers while maintaining sufficient capacity.
Explain physical-to-virtual (P2V) migration and server consolidation.
Estimate the total CPU and memory demand using the stated server specifications and average utilization. State all assumptions.
Assume a target average CPU utilization of 65% on the consolidated hosts.
Calculate the minimum number of hosts required based on CPU capacity, then check whether memory capacity changes the result.
Include a reasonable capacity reserve for peak demand and host failure; show the calculation clearly.
Discuss resource overcommitment, CPU scheduling, memory ballooning and the noisy-neighbour problem.
Evidence Expected: Worked calculations showing CPU and memory requirements, assumptions, capacity reserve and the recommended number of physical hosts. A final number without working will not receive full credit.

TASK 4 – Compare Virtual Machines and Containers (4 Marks)

Determine which workloads should run in virtual machines and which can run in containers.
Explain the architecture and characteristics of virtual machines and containers.

Compare VM and container isolation, operating-system requirements, startup time, resource overhead, portability and management.
Map at least five university workloads (for example, database, legacy student system, web portal, learning platform and analytics service) to VMs or containers.
Explain how Docker and a container orchestration platform such as Kubernetes can support deployment and scaling.
Identify situations where a VM is more appropriate than a container.
Evidence Expected: A VM-versus-container comparison and a workload mapping table supported by technical justification.

TASK 5 – Design Virtualized Storage and Network Infrastructure (4 Marks)

Design storage and network resources that support the proposed virtualized data centre.
Explain storage virtualization and compare direct-attached storage (DAS), network-attached storage (NAS) and storage area networks (SAN).
Explain software-defined storage (SDS), thin provisioning, snapshots, replication and backup.
Explain network virtualization, virtual switches, VLANs, virtual networks and software-defined networking (SDN).
Propose separate logical network segments for administration, student services, databases and management traffic.
Recommend suitable storage and network approaches for performance, security and availability.
Evidence Expected: A labelled storage and network design with explanations of the selected technologies and their roles.

TASK 6 – Plan Resource Allocation, Scheduling and Monitoring (5 Marks)

Develop a resource-management strategy that allocates infrastructure capacity efficiently.
Explain CPU, memory, storage and network resource allocation to VMs and containers.
Compare static allocation and dynamic allocation, including resource reservations, limits and shares.
Explain load balancing, workload scheduling, auto-scaling and high availability.
Identify suitable performance indicators: CPU utilization, memory utilization, disk I/O, network throughput, latency and VM availability.
Propose monitoring and alerting methods, and explain how the administrator should respond to resource contention.
Evidence Expected: A resource-allocation policy table and a monitoring plan that includes measurable indicators, thresholds and corrective actions.

TASK 7 – Evaluate Data Centre Availability, Security and Disaster Recovery (4 Marks)

Identify the risks associated with a virtualized data centre and propose suitable controls.

Explain hypervisor security, VM isolation, secure VM images, patch management and privileged access control.
Discuss VM escape, VM-to-VM attacks, insecure snapshots, VM sprawl and denial-of-service risks.
Recommend network segmentation, encryption, multi-factor authentication, role-based access control and audit logging.
Explain backup, replication, snapshots, live migration and disaster recovery.
Define Recovery Time Objective (RTO) and Recovery Point Objective (RPO), and propose suitable targets for at least two university applications.
Evidence Expected: A risk-and-mitigation table and a disaster-recovery plan with justified RTO and RPO values.

TASK 8 – Create the Virtualized Cloud Data Centre Architecture (5 Marks)

Construct an end-to-end architecture integrating compute, storage, network, virtualization and management components.
Draw a labelled architecture showing users, internet or campus network, firewall, load balancer, physical hosts, hypervisor layer, VMs/containers, virtual network, storage and monitoring.
Show how the university applications are distributed across hosts and how shared resources are allocated.
Include a management plane for provisioning, monitoring, logging and policy enforcement.
Show backup or replication to a secondary site or cloud region.
Justify the architecture in terms of scalability, performance, fault tolerance, security and cost.
Evidence Expected: A complete, clearly labelled architecture diagram and a written justification for each major component and design decision.

TASK 9 – Hands-on Implementation and Validation (5 Marks)

Build a small proof-of-concept that demonstrates virtualization and resource management.
Create at least one VM using VirtualBox, VMware, Hyper-V or KVM, or create a container using Docker.
Deploy a simple university-related service, such as a web page, student-record API or database-backed application.
Configure CPU, memory and storage resources and record the selected values.
Perform functional tests and at least one resource or load test. Record the observations.
Capture screenshots of the VM/container configuration, running service, resource-monitoring output and test results.
Document installation steps, commands, configuration details, issues faced and how they were resolved.
Evidence Expected: Reproducible implementation steps, source/configuration files, annotated screenshots and actual test evidence. Do not present simulated results as actual execution.

TASK 10 – Evaluate Cost, Performance and Final Recommendation (3 Marks)

Evaluate the proposed solution against the existing physical data-centre arrangement.

Compare the current physical-server model and proposed virtualized model in terms of hardware cost, electricity, cooling, maintenance, licensing and administration effort.

Prepare a simple three-year cost comparison using clearly stated assumptions.

Separate one-time capital expenditure from recurring operational expenditure.

Evaluate expected improvements in utilization, scalability, availability and manageability.

Identify risks such as licensing costs, vendor lock-in, skills requirements, hardware failure and migration downtime.

Provide a phased migration roadmap, identifying which applications should be migrated first and why.

Evidence Expected: A three-year cost table, risk analysis, migration roadmap and evidence-based final recommendation.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Virtualized Infrastructure (8 Marks)

Changed Requirement

The university introduces a digital learning platform and extends selected services to 10 affiliated colleges. The expected peak workload increases to five times the current peak. The system must achieve 99.9% availability, provide secure isolation between colleges and remain within a limited operating budget.

Re-evaluate your original design. You may RETAIN, MODIFY or REDESIGN the proposed infrastructure.

Impact Analysis – 2 Marks

Identify changes in workload, users, storage, network traffic and availability requirements.

Explain how multi-tenant access affects VM/container isolation, network segmentation and access control.

Identify which original capacity and cost assumptions are no longer valid.

Capacity and Availability Evaluation – 2 Marks

Estimate the additional CPU, memory, storage and network capacity needed for the five-times peak workload.

Evaluate whether the proposed number of hosts and the existing failover arrangement are sufficient.

Explain the meaning of 99.9% availability (approximately 8.76 hours of downtime per year) and identify measures needed to approach it.

Adaptation or Redesign – 2 Marks

Decide whether to retain, modify or redesign the original infrastructure and justify the decision.

Describe the revised architecture. Consider clustering, live migration, auto-scaling, container orchestration, multi-zone deployment or hybrid-cloud capacity where appropriate.

Explain changes to resource-allocation policies, tenant isolation, backup and disaster recovery.

Evidence-Based Defence – 2 Marks

Support the revised design with capacity calculations and clearly stated assumptions.

Compare the cost and operational complexity of the original and revised designs.

Discuss trade-offs involving performance, availability, security, licensing and management effort.

Evidence Expected: A technically justified decision supported by revised architecture, capacity calculations, availability considerations, security controls and cost comparison.

.Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks

Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load
- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks

Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)

- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks

Discuss:

- Capacity and availability calculations
- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

MARK DISTRIBUTION

Task	Title	Bloom's Level	Marks
Task 1	Existing Data Centre and Requirements	L2 – Understand / Analyse	3
Task 2	Virtualization Fundamentals and Hypervisors	L2 – Understand	3
Task 3	Server Consolidation and Resource Calculations	L3 – Apply	3
Task 4	Virtual Machines and Containers	L4 – Analyse	5
Task 5	Virtualized Storage and Network Infrastructure	L3/L4 – Apply / Analyse	4
Task 6	Resource Allocation, Scheduling and Monitoring	L4 – Analyse	5
Task 7	Availability, Security and Disaster Recovery	L4 – Analyse	3
Task 8	Virtualized Cloud Data Centre Architecture	L6 – Create	5
Task 9	Hands-on Implementation and Validation	L3 – Apply	4
Task 10	Cost, Performance and Final Recommendation	L5 – Evaluate	7
	Subtotal		

Task 11	Day-4 Design Challenge	L4/L5/L6 – Analyse / Evaluate / Create	8
	TOTAL		50

TASK 1 – ANALYSE THE EXISTING DATA CENTRE AND IDENTIFY REQUIREMENTS

1.1 Purpose and Components of a Data Centre

A data centre is a centralized facility that contains computing, storage and networking equipment used to store, process, manage and deliver an organization's applications and data. The university data centre supports the Student Information System (SIS), Learning Management System (LMS), Library Management System, Examination Portal, research applications and institutional website.

Component	Purpose in the university
Compute	Physical servers and processors run university applications and databases.
Storage	Disks and storage systems store student records, examination data, documents and backups.
Networking	Switches, routers and firewalls connect users, servers, the internet and campus networks.
Power	UPS, power distribution units and generators provide continuous electrical supply.
Cooling	Air-conditioning and precision cooling prevent servers from overheating.
Management	Monitoring, configuration, access control, backup and administration tools manage the facility.

1.2 Limitations of the Existing Environment

Existing problem	Effect	Virtualization requirement
Low average CPU utilization (20%)	Most server capacity remains unused.	Combine workloads on shared hosts through server consolidation.
Peak CPU utilization of 90–95%	Applications may become slow during admissions and examinations.	Use resource scheduling, reservations and elastic capacity.

One application per physical server	Resources cannot be shared easily.	Create multiple isolated VMs on common physical hosts.
High electricity and cooling cost	More physical servers consume more power and cooling.	Reduce the number of physical machines where capacity permits.
Slow hardware procurement	New applications take time to deploy.	Provision VMs and containers quickly from templates.
No centralized VM management or tested migration	Maintenance and failures cause service disruption.	Use centralized management, clustering and live migration.
No separate tested disaster-recovery site	Data and services may remain unavailable after a major failure.	Use backups, replication and a documented DR plan.

1.3 Important Definitions

Resource utilization: The percentage of available CPU, memory, storage or network capacity currently being used.

Server consolidation: Combining workloads from several underutilized physical servers onto fewer, more powerful hosts using virtualization.

Scalability: The ability of a system to increase or decrease resources according to workload demand.

Availability: The percentage of time a service is operational and accessible to users.

Fault tolerance: The ability of a system to continue operating when a component fails, with little or no interruption.

1.4 Requirement Table

Requirement	University need	Proposed solution
Performance	Fast access during admissions, examinations and results.	Load balancing, CPU scheduling, SSD storage and monitoring.

Capacity	Support current applications and future digital learning.	Pooled compute, memory and storage with expansion capacity.
Security	Protect student, staff and examination information.	Network segmentation, encryption, MFA and role-based access.
Availability	Keep important portals operational.	Host clustering, redundant networking, backups and DR replication.
Cost	Reduce power, cooling and maintenance expenses.	Server consolidation, automation and measured cloud bursting.

Conclusion:

The current environment has low resource utilization, high operating cost and inadequate recovery arrangements. A virtualized infrastructure is required to pool resources, improve utilization, deploy applications faster and strengthen availability and disaster recover.

TASK 2 – EXPLAIN VIRTUALIZATION FUNDAMENTALS AND HYPERVISORS

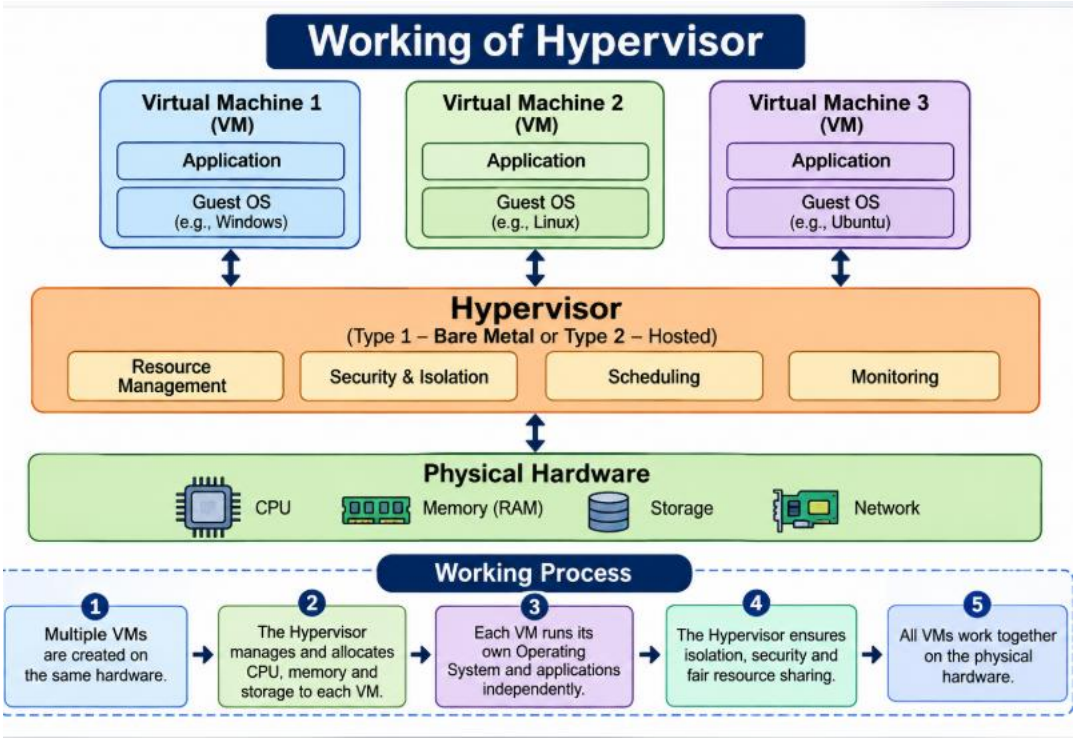
2.1 Definition and Purpose

Virtualization is the process of creating virtual versions of computing resources such as servers, operating systems, storage and networks. It allows multiple isolated virtual machines to share one physical server. In a cloud data centre, virtualization improves hardware utilization, flexibility, scalability, isolation and centralized management.

2.2 Role and Working of a Hypervisor

A hypervisor, also called a Virtual Machine Monitor (VMM), is software or firmware that creates and manages virtual machines. It allocates CPU, memory, storage and network resources to each VM, keeps VMs isolated and controls access to physical hardware.

Physical server hardware (CPU, RAM, disk, network)



2.3 Type-1 and Type-2 Hypervisors

Feature	Type-1 (Bare-metal)	Type-2 (Hosted)
Installation	Runs directly on physical hardware.	Runs as an application on a host operating system.
Performance	Usually higher; fewer software layers.	Has additional host-OS overhead.

Use	Enterprise servers and data centres.	Learning, testing and desktop virtualization.
Examples	VMware ESXi, Microsoft Hyper-V, KVM.	Oracle VirtualBox, VMware Workstation.

Recommended choice: A Type-1 hypervisor such as VMware ESXi or a properly managed KVM-based platform is suitable for the university production data centre. It provides efficient hardware access, centralized management, VM isolation and support for clustering and live migration. Oracle VirtualBox is suitable for the student proof-of-concept, not as the main production platform.

2.4 Virtualization Techniques

Technique	Meaning	Example / use
Full virtualization	The hypervisor presents virtual hardware so an unmodified guest OS can run.	Run Windows or Linux as a VM.
Para-virtualization	The guest OS is aware of virtualization and cooperates with the hypervisor through optimized interfaces.	Optimized virtual I/O drivers and paravirtualized devices.
Hardware-assisted virtualization	CPU features such as Intel VT-x or AMD-V help execute guest operating systems efficiently.	Modern server virtualization.
OS-level virtualization	Multiple isolated containers share the host OS kernel.	Docker containers for web services.

Conclusion:

Virtualization allows the university to run several isolated workloads on shared physical infrastructure. A Type-1 hypervisor is recommended for production, while Type-2 software can be used for laboratory demonstrations.

TASK 3 – DESIGN SERVER CONSOLIDATION AND CALCULATE RESOURCE REQUIREMENTS

3.1 P2V Migration and Server Consolidation

Physical-to-Virtual (P2V) migration converts an existing physical server and its operating system, applications and data into a virtual machine. Server consolidation places multiple migrated VMs on a smaller pool of physical hosts. Before migration, compatibility, application dependencies, storage, network settings and backup must be checked.

3.2 Existing Resource Calculation

Given: 20 servers; each has 16 CPU cores, 64 GB RAM and 2 TB storage.

Total CPU capacity = 20 × 16 = 320 CPU cores

Average CPU demand = 320 × 20% = 64 cores

Peak CPU demand (using 95%) = 320 × 95% = 304 cores

Total physical memory = 20 × 64 = 1,280 GB

Assumed average memory use = 1,280 × 60% = 768 GB

Total local storage = 20 × 2 = 40 TB

3.3 Host Requirement at 65% Target Average CPU Utilization

Assume each proposed host has 32 CPU cores and 512 GB RAM. At a target CPU utilization of 65%, one host provides 32 × 0.65 = 20.8 cores of planned CPU demand.

CPU hosts for average demand = 64 / 20.8 = 3.08

Round up = 4 hosts (without failure reserve)

For N+1 host failure protection, 4 hosts are not sufficient:

3 surviving hosts × 32 × 0.65 = 62.4 cores < 64 cores.

5 hosts: after one host fails, 4 × 32 × 0.65 = 83.2 cores.

Therefore, 5 hosts meet average demand with one-host failure reserve.

Memory check: 5 hosts provide 5 × 512 = 2,560 GB. After one host fails, 4 × 512 = 2,048 GB, which is greater than the assumed 768 GB average memory demand. CPU is the limiting resource for average-load planning.

3.4 Peak-Demand Capacity Check

Peak CPU demand = 304 cores

Required host count at 65% target = 304 / (32 × 0.65)

= 304 / 20.8 = 14.62

Round up = 15 hosts without N+1 reserve.

With one host failure: 16 hosts are required.

After one failure: 15 × 32 × 0.65 = 312 cores,

which is greater than 304 cores.

Recommended planning: Five hosts can support the calculated average demand with one-host failure protection, but they cannot support the stated 90–95% peak demand. If the university requires all existing peak demand to be handled locally at the 65% planning target, it should plan for 16 hosts, subject to workload measurement and budget approval. A practical phased design can begin with a smaller cluster and add hosts before peak periods, while using cloud bursting only after testing.

3.5 Resource Management Concepts

Resource overcommitment: Assigning more virtual CPU or memory than the physical capacity, based on the expectation that all VMs will not use their maximum at the same time. Excessive overcommitment can cause performance problems.

CPU scheduling: The hypervisor shares processor time among VMs according to shares, reservations, limits and workload priority.

Memory ballooning: A hypervisor driver reclaims unused memory from VMs when the host needs memory. It should be monitored because excessive reclamation can affect performance.

Noisy-neighbour problem: One VM or tenant consumes excessive shared resources and reduces performance for other VMs. Resource limits, reservations and monitoring help control it.

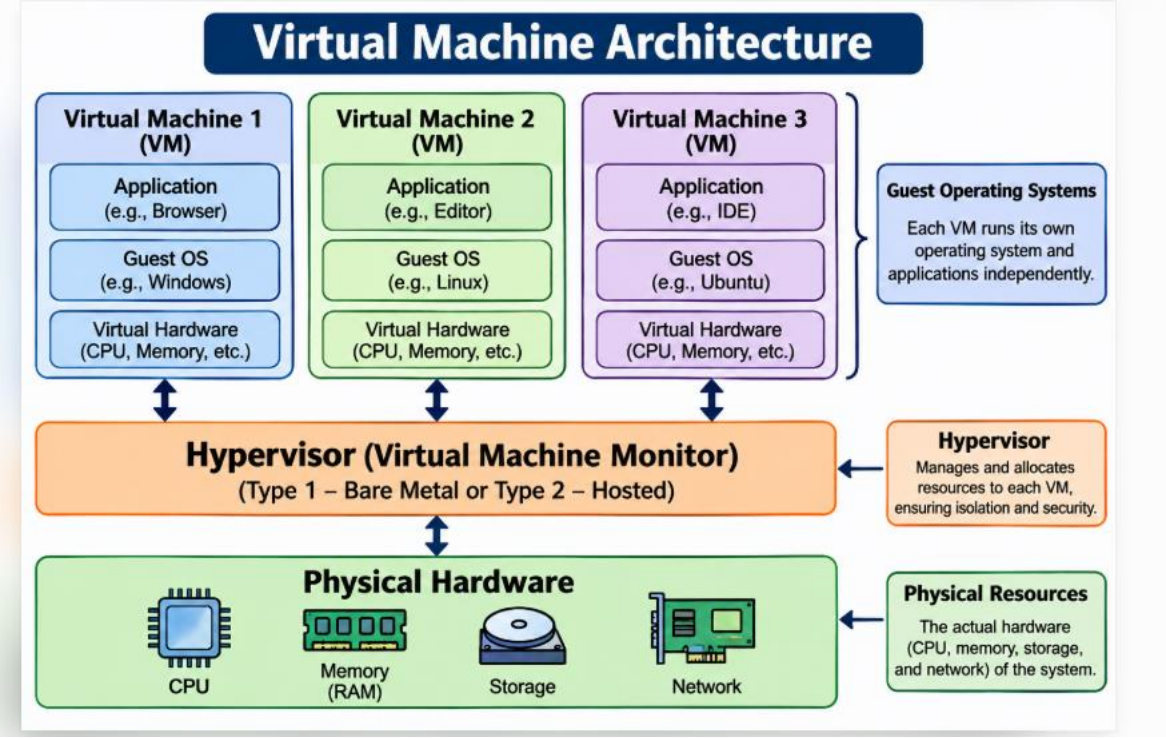
Conclusion:

Resource calculations must consider both average and peak demand. A five-host cluster is adequate for average CPU demand with N+1 protection under the stated assumptions; sixteen 32-core hosts are needed to meet the calculated current peak at a 65% target with one host failure.

TASK 4 – COMPARE VIRTUAL MACHINES AND CONTAINERS

4.1 Virtual Machine Architecture

A virtual machine is a software-based computer with virtual CPU, memory, disk and network interfaces. Each VM runs its own guest operating system on a hypervisor. VMs provide strong separation between operating-system environments and are useful for legacy or different-OS applications.



4.2 Container Architecture

A container packages an application and its libraries and dependencies into an isolated unit. Containers share the host operating-system kernel but have separate process, filesystem and network namespaces. Docker is commonly used to build and run containers.

4.3 VM and Container Comparison

Feature	Virtual Machine	Container
Isolation	Separate guest OS and virtual hardware; generally stronger OS-level isolation.	Shares host kernel; process and namespace isolation.
Operating system	Each VM can run its own guest OS.	Must use a compatible host kernel family.
Startup time	Usually slower because a full OS must boot.	Usually seconds or less.
Resource overhead	Higher due to guest OS and virtual hardware.	Lower; shares host OS kernel.
Portability	VM images are portable between compatible hypervisors.	Container images are portable between compatible container runtimes.
Management	Managed through hypervisor and VM tools.	Managed through container runtime and orchestration tools.
Best use	Legacy applications, databases needing OS control, different OS requirements.	Stateless web services, APIs, microservices and repeatable deployments.

4.4 University Workload Mapping

University workload	Recommended platform	Reason
Legacy Student Information System	VM	May require a specific operating system, libraries or older software.
Student database	VM	Needs controlled storage, memory, backup and database administration; can be containerized only after validation.

Institutional web portal	Container	Web application can be packaged and replicated easily.
Digital Learning Platform services	Containers	Independent services can scale separately using orchestration.
Analytics and report-generation service	Containers or VM	Containers suit repeatable batch services; VM is appropriate if special OS or hardware dependencies exist.
Examination application	VM or dedicated isolated container group	Requires strict security, controlled access and predictable resources.

4.5 Docker and Kubernetes

Docker builds an image containing an application and its dependencies, then runs that image as a container.

A container registry stores versioned images so deployment can be repeated consistently.

Kubernetes orchestrates containers across multiple hosts. It supports scheduling, service discovery, health checks, restart of failed containers, rolling updates and horizontal scaling.

Horizontal Pod Autoscaler can increase or decrease the number of application pods based on measured resource demand.

4.6 When a VM Is More Appropriate

When the application needs a different guest operating system or kernel.

When a legacy application depends on OS-specific drivers or configuration.

When stronger isolation boundaries are required between sensitive workloads.

When the application needs direct control over OS patching, kernel settings or specialized software.

Conclusion: The university should use VMs as the secure base for legacy systems and databases, and containers for modern web applications, APIs and independently scalable services. Kubernetes can manage container deployment and scaling.

TASK 5 – DESIGN VIRTUALIZED STORAGE AND NETWORK INFRASTRUCTURE

5.1 Storage Virtualization

Storage virtualization combines physical storage devices from multiple systems into a logical storage pool. Administrators can allocate virtual disks to VMs without requiring each VM to be tied to one physical disk.

Storage type	Description	University use
DAS	Direct-Attached Storage is connected directly to one server.	Local boot disks or temporary high-speed storage; less convenient for shared failover.
NAS	Network-Attached Storage provides file-level storage over a network.	Shared documents, user files and departmental file shares.
SAN	Storage Area Network provides block-level storage to servers over a dedicated high-speed network.	Shared VM datastores and critical databases where performance and multipath access are required.

5.2 Important Storage Technologies

Software-Defined Storage (SDS): Storage capacity is pooled and managed through software policies rather than being tied to one storage appliance.

Thin provisioning: Logical storage is allocated to a VM on demand, instead of reserving the entire maximum size immediately. Actual free capacity must be monitored.

Snapshots: Point-in-time records of a VM or volume that help with short-term rollback. A snapshot is not a replacement for an independent backup.

Replication: Copies data to another storage system or site to support recovery after a failure.

Backup: Creates recoverable copies of data, preferably with offline or immutable protection and periodic restore tests.

5.3 Network Virtualization

Network virtualization creates logical networks on top of physical switches and network links. A virtual switch connects VMs and containers. VLANs separate traffic at Layer 2, while virtual networks and software-defined networking (SDN) allow centrally managed network policies.

Network segment	Purpose and access rule
Administration VLAN	Restricted access for authorized IT administrators using MFA and secure management protocols.
Student services VLAN	Provides access to SIS, LMS and public student-facing portals; cannot directly access database management ports.
Database VLAN	Allows only approved application servers to connect to database ports; blocks direct student access.
Management VLAN	Carries hypervisor, storage and monitoring traffic; isolated from public user networks.
Backup/replication network	Carries backup and replication traffic with restricted routes and encryption.

5.4 Recommended Design

- Use shared SAN or a reliable SDS cluster for VM disks and critical databases, with redundant controllers and multipath connectivity.
- Use NAS for shared documents, departmental files and user content.
- Use SSD-backed storage tiers for databases and high-I/O services; use lower-cost capacity tiers for archives and backups.
- Use redundant physical switches, virtual switches, VLAN segmentation and firewall rules between security zones.
- Encrypt data in transit and at rest, restrict management interfaces and monitor storage capacity and network latency.

Conclusion:

A combination of SAN/SDS for critical VM storage and NAS for file sharing provides flexibility. Segmented virtual networks protect sensitive databases and management traffic.

TASK 6 – PLAN RESOURCE ALLOCATION, SCHEDULING AND MONITORING

6.1 Resource Allocation

CPU allocation assigns virtual processors to VMs and CPU requests/limits to containers. Memory allocation defines the amount of RAM available to a workload. Storage allocation provides virtual disks, volumes and IOPS capacity. Network allocation controls bandwidth, virtual interfaces and traffic policies.

6.2 Static and Dynamic Allocation

Method	Meaning	Advantages / limitations
Static allocation	Fixed CPU, memory or storage assigned in advance.	Predictable, but may waste resources during low demand.
Dynamic allocation	Resources are adjusted according to demand and policies.	Improves utilization and responsiveness; requires monitoring and safe limits.
Reservation	Minimum resource capacity guaranteed to a VM.	Protects critical services during contention.
Limit	Maximum resource usage allowed.	Prevents one workload from consuming all resources.
Shares	Relative priority when resources are contended.	Useful for prioritizing examination and student services.

6.3 Scheduling and Scaling

Load balancing distributes incoming requests across multiple healthy application instances.
Workload scheduling selects a suitable host or cluster node based on available CPU, memory, affinity and policy.
Auto-scaling increases or decreases VM instances, container replicas or cloud resources according to workload metrics.
High availability (HA) restarts or moves workloads after a host or component failure, subject to available spare capacity.

6.4 Resource Allocation Policy

Workload	CPU policy	Memory policy	Priority
SIS and examination portal	Reserve capacity; moderate limit; high shares.	Guaranteed baseline; avoid swapping.	Critical
LMS and digital learning	Dynamic scaling; minimum replicas.	Requests and limits per service.	High
Library system	Normal shares; scale during peak usage.	Baseline allocation with monitoring.	Medium
Analytics jobs	Schedule off-peak; apply CPU limits.	Limit batch memory use.	Normal
Development/testing	Low shares and strict limits.	Small fixed limits.	Low

6.5 Monitoring Plan and Thresholds

Indicator	Warning threshold (example)	Corrective action
CPU utilization	Above 75% for 10 minutes; critical above 90%.	Identify top consumers, rebalance VMs, scale out or add hosts.
Memory utilization	Above 80% sustained; critical above 90%.	Check leaks, ballooning and swapping; increase memory or replicas.
Disk capacity	Above 80% used; critical above 90%.	Expand datastore, archive old data and enforce retention.
Disk I/O latency	Above 20 ms sustained for critical datastore (tune to workload).	Check queue depth, noisy neighbours and storage health.
Network throughput	Above 75% link capacity sustained.	Inspect traffic, apply QoS or increase link capacity.

Application latency	Above agreed service objective, e.g. 2 seconds for common portal requests.	Check application, database, network and load balancer.
VM/service availability	Any failed health check or repeated restart.	Alert administrator; fail over or restart and investigate.

Monitoring tools may include the hypervisor management console, Prometheus and Grafana for metrics, centralized syslog for logs, and application health checks. Alerts should reach the IT operations team through email or an incident-management system.

6.6 Response to Resource Contention

Confirm the alert and identify the affected host, VM, container or application.
Check CPU ready time, memory pressure, disk latency, network drops and application logs.
Prioritize critical services using reservations and shares; limit non-critical batch jobs.
Move workloads, add replicas or provision additional capacity if required.
Record the incident and review capacity trends to prevent recurrence.
Conclusion: A combination of reservations, limits, shares, load balancing and auto-scaling helps the university use resources efficiently while protecting critical applications.

TASK 7 – EVALUATE DATA CENTRE AVAILABILITY, SECURITY AND DISASTER RECOVERY

7.1 Security Risks and Controls

Risk	Explanation	Mitigation
Hypervisor attack	An attacker targets the virtualization layer.	Patch hypervisors, restrict management access and use secure configuration.
VM escape	A compromised VM attempts to access the host or other VMs.	Use supported hypervisors, patch regularly and apply strong isolation.
VM-to-VM attack	A compromised VM attacks another VM on a shared network.	Use VLANs, micro-segmentation and firewall rules.
Insecure snapshots	Snapshots may contain sensitive data or credentials.	Encrypt, restrict access and delete snapshots according to policy.
VM sprawl	Unused VMs consume capacity and may remain unpatched.	Use an inventory, owner approval, expiry dates and periodic audits.
Denial of Service	Excessive traffic or resource consumption makes services unavailable.	Use rate limits, DDoS protection, quotas and load balancing.

7.2 Security Practices

- Use multi-factor authentication (MFA) for administrators.
- Apply role-based access control (RBAC) and least-privilege permissions.
- Encrypt sensitive data at rest and data in transit.
- Use secure, patched VM templates and scan container images before deployment.
- Maintain audit logs for administrator actions, VM changes and access to sensitive records.
- Separate management, student, database and backup networks.

7.3 Backup and Disaster Recovery

A snapshot captures a point-in-time state and is useful for short-term rollback. A backup is an independent recoverable copy. Replication copies data to another system or site. Live migration moves a running VM between compatible hosts with minimal interruption; it is mainly for maintenance and load balancing, not a substitute for backup or disaster recovery. Recovery Time Objective (RTO) is the maximum acceptable time to restore a service after disruption. Recovery Point Objective (RPO) is the maximum acceptable amount of data loss measured in time.

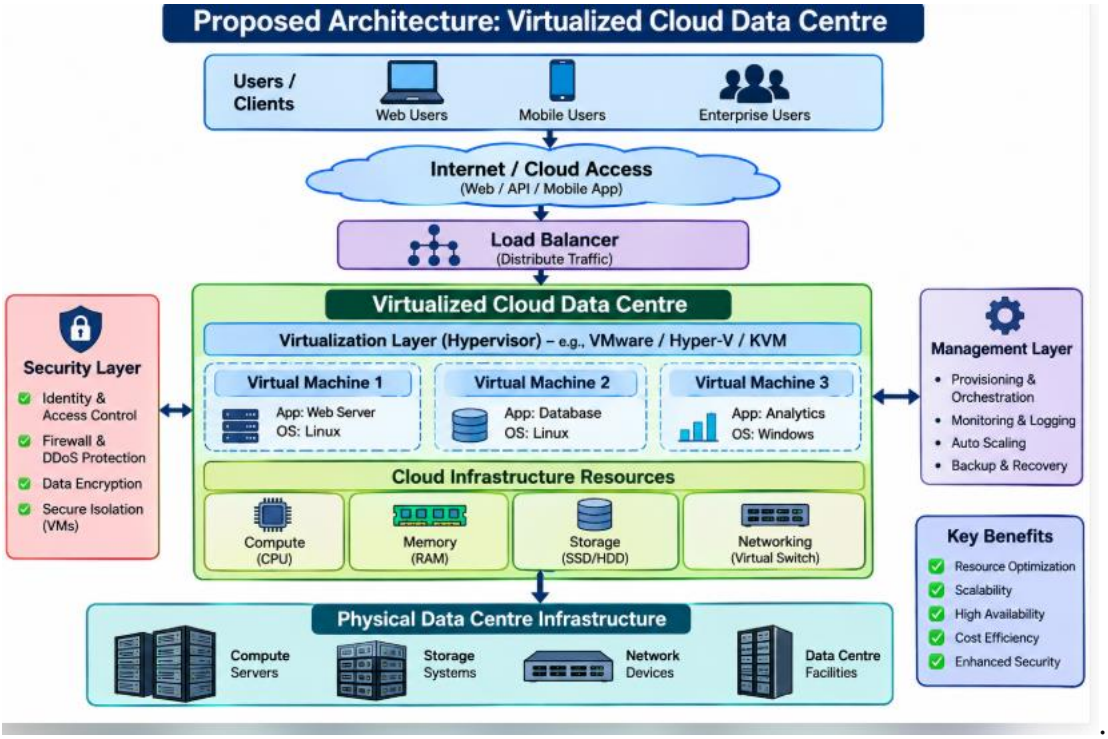
Application	Proposed RTO	Proposed RPO	Reason
Student Information System	2 hours	15 minutes	Student registration and academic records are critical.
Learning Management System	4 hours	30 minutes	Learning access should resume quickly, with limited loss of recent activity.

Disaster-recovery plan: Maintain encrypted daily backups and frequent transaction-log backups for databases; replicate critical data to a secondary site or cloud region; keep immutable/offline backup copies; document recovery order and contacts; test restoration at least twice a year; and record actual RTO/RPO achieved.

Conclusion: Security requires layered controls across hypervisors, VMs, networks, identities and data. Tested backups, replication and recovery procedures reduce the impact of hardware failure, cyberattacks and site disasters.

TASK 8 – CREATE THE VIRTUALIZED CLOUD DATA CENTRE ARCHITECTURE

8.1 Proposed Architecture Diagram



8.2 Component Explanation

Component	Role and justification
Users and campus network	Students, faculty, administrators and affiliated users access services through controlled entry points.
Firewall and DDoS protection	Filters unauthorized traffic and helps protect public services from malicious traffic.
Load balancer	Distributes requests across healthy web/API instances and improves availability.
Physical hosts and Type-1 hypervisor	Provide pooled CPU and memory; VMs isolate operating systems and critical workloads.

Containers and orchestration	Run modern web applications and APIs; support replica scaling and recovery.
Virtual network / VLAN / SDN	Separates student, administration, database, management and backup traffic.
SAN/SDS and NAS	Provide shared block storage for VM workloads and file storage for documents.
Management plane	Centralizes provisioning, monitoring, logging, identity, configuration and policy enforcement.
Secondary DR site	Stores replicated data and recoverable services for major site failures.

8.3 Deployment and Service Models

Private cloud: The university operates the virtualized infrastructure for institutional workloads requiring control and data governance.
Hybrid cloud: Selected services or temporary peak capacity may use a public cloud after security, compliance and cost evaluation.
IaaS: Provides virtual machines, networks and storage for legacy systems and databases.
PaaS: Provides managed application runtime, databases or deployment platforms for developers.
SaaS: Provides ready-to-use services such as email, collaboration or learning tools where appropriate.

8.4 Scaling, Security and Disaster Recovery

Scale vertically by increasing CPU or memory for a VM when supported, and horizontally by adding web/API replicas behind the load balancer.
Use cluster capacity and live migration for maintenance and host failure handling.
Enforce least privilege, MFA, encryption, image scanning and network segmentation.
Replicate critical databases and maintain independent backups at a secondary site.

Conclusion:

The architecture combines a private virtualized cluster, containers for scalable services, shared storage, segmented virtual networking and a secondary recovery location. It improves manageability, resource sharing and resilience.

TASK 9 – HANDS-ON IMPLEMENTATION AND VALIDATION

9.1 AIM

To develop and deploy a simple University Student Record REST API using Python Flask and Docker. The application is tested to verify its functionality and demonstrate containerization with resource allocation.

9.2 REQUIREMENTS AND CONFIGURATION

Software Requirements:

1. Python 3
2. Flask framework
3. Docker Desktop or Docker Engine
4. Notepad or Visual Studio Code
5. Web Browser or Command Prompt

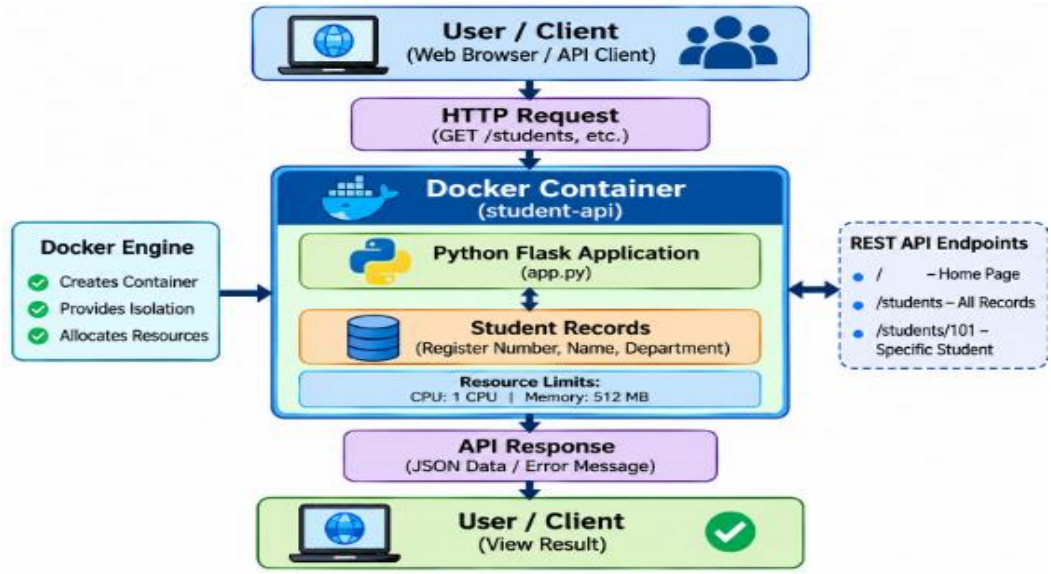
Resource Configuration:

Parameter	Configuration
CPU	1 CPU
Memory	512 MB
Port Number	5000
Programming Language	Python
Framework	Flask
Container Platform	Docker

The application provides sample student information through REST API endpoints.

9.3 PROPOSED IMPLEMENTATION ARCHITECTURE

Figure 1 – Docker-Based University Student API Architecture



Architecture Explanation:

The user sends an HTTP request through a web browser or API client. The request reaches the Flask application running inside a Docker container. The application processes the request and retrieves the required student information. Finally, the result is returned to the user in JSON format.

The Docker container provides an isolated environment for running the application with specified CPU and memory limits.

9.4 PROGRAM – app.py

Create a Python file named app.py and enter the following code.

```
from flask import Flask, jsonify

app = Flask(__name__)

students = [
    {"reg_no": "101", "name": "Arun", "department": "CSE"},
    {"reg_no": "102", "name": "Meena", "department": "CSE"}
]

@app.route("/")
def home():
    return "University Student API is running"
```

```
@app.route("/students")
def get_students():
    return jsonify(students)

@app.route("/students/<reg_no>")
def get_student(reg_no):
    for student in students:
        if student["reg_no"] == reg_no:
            return jsonify(student)
    return jsonify({"error": "Student not found"}), 404

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Program Explanation:

1. Flask is imported to create the REST API.
2. A list named students stores sample student details.
3. The / route displays the home page message.
4. The /students route displays all student records.
5. The /students/<reg_no> route searches for a particular student.
6. If the student is not found, the application returns a 404 error.

9.5 CREATE requirements.txt

Create another file named requirements.txt and enter the following:

```
Flask==3.0.3
```

This file specifies the Flask version required to run the application.

9.6 CREATE DOCKERFILE

Create a file named Dockerfile without any file extension.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Dockerfile Explanation:

1. FROM selects the Python base image.
2. WORKDIR creates the working directory.
3. COPY copies the application files.
4. RUN installs Flask.
5. EXPOSE specifies port 5000.
6. CMD starts the Flask application.

9.7 BUILD AND RUN COMMANDS

Save all three files in the same folder:

1. app.py
2. requirements.txt
3. Dockerfile

Open Command Prompt in that folder and execute the following commands.

Step 1: Build the Docker Image

docker build -t university-student-api .

This command creates a Docker image named university-student-api.

Step 2: Run the Docker Container

docker run -d --name student-api -p 5000:5000 --cpus="1.0" --memory="512m" university-student-api

This command starts the container with one CPU and 512 MB memory limit.

Step 3: Check the Running Container

docker ps

This command displays the running Docker containers.

Step 4: Monitor Resource Usage

docker stats student-api

This command displays the CPU and memory usage of the container.

9.8 TESTING THE APPLICATION

Open a web browser and enter the following URLs.

Test Case 1: Home Page

URL:

http://localhost:5000/

Expected Output:

University Student API is running

Test Case 2: Display All Student Records

URL:

http://localhost:5000/students

Expected Output:


```
[
  {
    "department": "CSE",
    "name": "Arun",
    "reg_no": "101"
  },
  {
    "department": "CSE",
    "name": "Meena",
    "reg_no": "102"
  }
]
```


Test Case 3: Search for an Existing Student

URL:

http://localhost:5000/students/101

Expected Output:


```
{
  "department": "CSE",
```

"name": "Arun",
"reg_no": "101"
}

Test Case 4: Search for a Missing Student

URL:

http://localhost:5000/students/999

Expected Output:

{
"error": "Student not found"
}

HTTP Status Code: 404

9.9 RESOURCE MONITORING AND VALIDATION

The Docker container is configured with a CPU limit of 1 CPU and a memory limit of 512 MB.

The following command is used to monitor the container's resource usage.

docker stats student-api

The command displays:

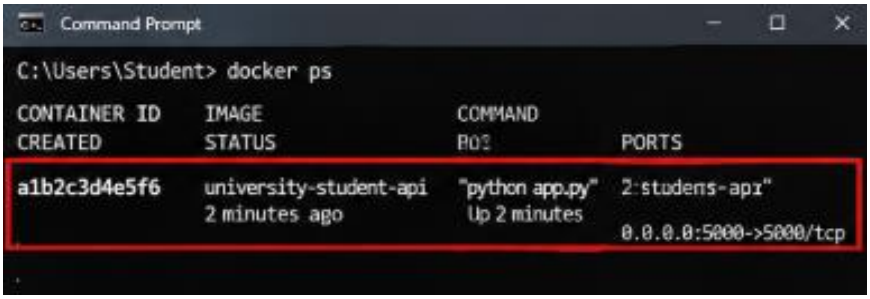
1. CPU usage.
2. Memory usage.
3. Network input and output.
4. Container resource statistics.

The application is validated by checking the following conditions:

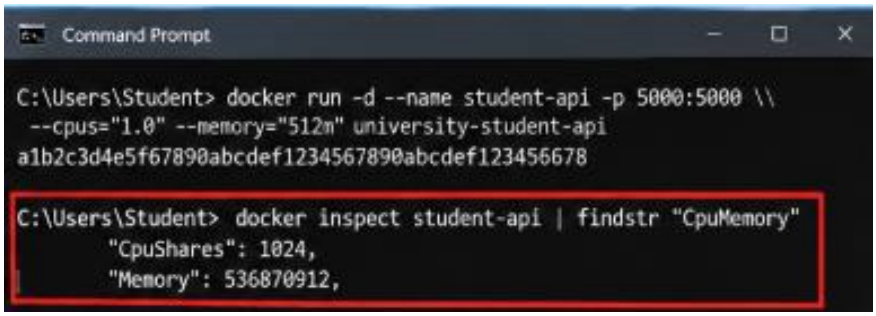
1. Docker container starts successfully.
2. Student records are displayed correctly.
3. Individual student details are retrieved.
4. Missing student requests return HTTP 404.
5. CPU and memory usage can be monitored.

9.10 SCREENSHOTS

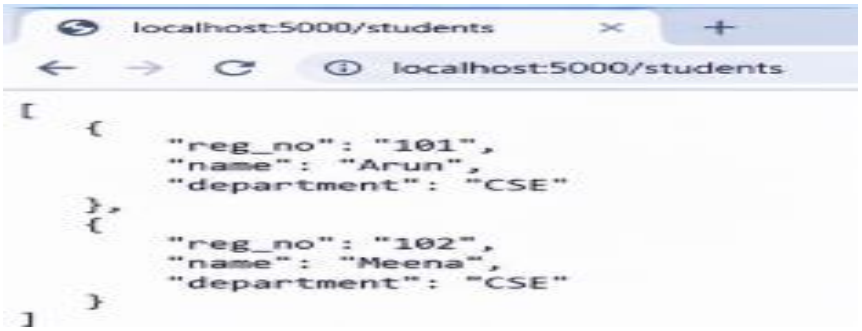
- Docker container running successfully using the docker ps command.



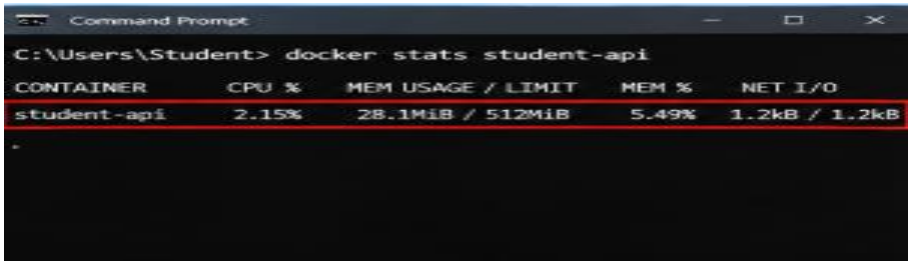
- Docker container configuration showing CPU limit of 1 CPU and memory limit of 512 MB.



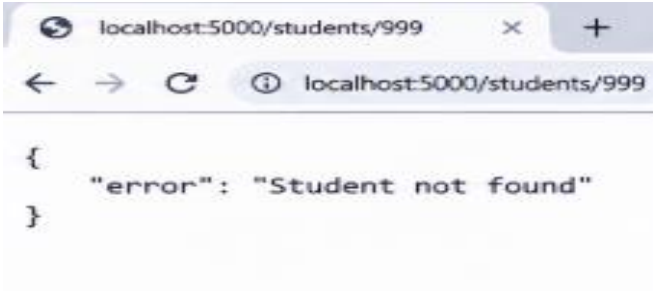
- Student records displayed in JSON format using the /students URL.



- CPU and memory usage displayed using the docker stats student-api command.



- Student-not-found error displayed using the /students/999 URL.



9.11 RESULT

Thus, a University Student Record REST API was developed using Python Flask and deployed using Docker. The application was tested for retrieving all student records, searching for individual students, and handling missing student requests. CPU and memory limits were configured, and resource usage was monitored using Docker. Hence, the implementation demonstrates containerization and resource management in a cloud computing environment.

TASK 10 – EVALUATE COST, PERFORMANCE AND FINAL RECOMMENDATION

10.1 Cost Assumptions

The following is an illustrative planning estimate in Indian Rupees (INR), not a vendor quotation. Actual costs depend on hardware brands, support contracts, electricity tariffs, software licensing and procurement discounts. Assume the existing 20-server environment is replaced by a 16-host peak-capable virtualized cluster, based on Task 3 peak calculation. Amounts are in lakh rupees (₹ lakh).

Cost item	Existing physical model	Virtualized model
Initial hardware / storage / network CAPEX	₹80 lakh	₹100 lakh
Annual electricity and cooling	₹18 lakh	₹12 lakh
Annual maintenance and support	₹10 lakh	₹8 lakh
Annual licensing and management	₹4 lakh	₹8 lakh
Annual administration effort (estimated)	₹8 lakh	₹6 lakh

10.2 Three-Year Cost Comparison

Existing model recurring cost per year = 18 + 10 + 4 + 8 = ₹40 lakh

Existing 3-year TCO = CAPEX 80 + (40 × 3) = ₹200 lakh

Virtualized model recurring cost per year = 12 + 8 + 8 + 6 = ₹34 lakh

Virtualized 3-year TCO = CAPEX 100 + (34 × 3) = ₹202 lakh

Cost measure	Existing physical model	Virtualized model
One-time CAPEX	₹80 lakh	₹100 lakh
Annual OPEX	₹40 lakh	₹34 lakh
Three-year OPEX	₹120 lakh	₹102 lakh
Three-year total cost	₹200 lakh	₹202 lakh

Interpretation: Under these illustrative assumptions, the virtualized design costs ₹2 lakh more over three years because of higher initial infrastructure and licensing expenditure. However, its annual operating cost is ₹6 lakh lower. The simple payback on the additional ₹20 lakh CAPEX is about 20/6 = 3.33 years, so the financial case becomes stronger over a longer period or if utilization, outage reduction and deployment speed create additional savings. A detailed procurement quotation is required before approval.

10.3 Performance and Operational Comparison

Factor	Existing physical model	Proposed virtualized model
Utilization	Average CPU utilization about 20%.	Pooled resources can be allocated to workloads according to demand.
Scalability	Requires purchase and installation of another physical server.	VMs and containers can be provisioned faster; cluster can be expanded.
Availability	No tested live migration or defined DR site.	Clustering, live migration, replication and tested DR plan.
Management	Separate server administration.	Centralized provisioning, monitoring and policy management.
Power and cooling	More separate servers and associated cooling.	Potential reduction through consolidation, subject to peak capacity.
Deployment	Hardware procurement delays.	Templates and container images support repeatable deployment.

10.4 Risks and Controls

Risk	Control

Virtualization licensing cost	Compare licensing models, open-source options and support costs before procurement.
Vendor lock-in	Use portable VM formats, standard APIs, container images and documented exit plans.
Skills gap	Train administrators in hypervisors, networking, storage, containers and security.
Hardware failure	Use N+1 capacity, redundant power/network paths and spare parts.
Migration downtime	Pilot low-risk systems, schedule maintenance windows, test rollback and verify data.
Performance contention	Use reservations, limits, shares, monitoring and capacity planning.

10.5 Phased Migration Roadmap

Phase	Activities	Reason
Phase 1 – Assessment	Inventory servers, dependencies, utilization, licenses, data and recovery needs.	Avoids migration surprises and identifies suitable workloads.
Phase 2 – Build foundation	Install hypervisor cluster, shared storage, redundant network, identity and monitoring.	Creates a secure platform before moving applications.
Phase 3 – Pilot	Migrate development, test and low-risk web services; validate performance and backup.	Tests the process with limited business risk.
Phase 4 – Core applications	Migrate library and institutional website, then	Moves workloads in order of complexity and business criticality.

	SIS and LMS in planned windows.	
Phase 5 – Optimize	Tune resource allocations, remove unused servers, test failover and DR.	Confirms benefits and improves capacity utilization.

10.6 Final Recommendation

The university should adopt a phased virtualized private-cloud design using a Type-1 hypervisor cluster, shared storage, segmented virtual networking and containers for modern web services. The peak-capacity calculation indicates that a 16-host, 32-core design is needed to support the stated current peak at a 65% planning target with one-host failure reserve, under the assumptions in Task 3. Procurement should be staged and supported by workload measurements. The financial estimate shows that savings in annual operating cost may take more than three years to recover the higher initial investment; therefore, licensing, support and hardware quotations must be compared before final approval.

TASK 11 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE INFRASTRUCTURE

Changed requirement: The university introduces a digital learning platform and extends selected services to affiliated colleges. Peak workload increases to five times the current peak. The service must achieve at least 99.9% availability, isolate tenants securely and remain within a limited operating budget. The question also refers to 10 affiliated colleges in the scenario and a 40-college isolation point in the later evidence prompts. The design below uses 40 tenants for the stricter isolation case; if the final scope is only 10 colleges, the same model applies with fewer tenant configurations.

11.1 Impact Analysis (2 Marks)

- **Workload:** Digital learning, video/content access, assessments and API traffic increase peak CPU demand to five times the current peak.

- **Users:** Students, faculty and administrators from the university and affiliated colleges access shared services concurrently.

- **Storage:** Learning content, submissions, attendance and assessment records increase storage and backup requirements.

- **Network:** More concurrent sessions, file downloads, video traffic and API calls increase bandwidth and load-balancer demand.

- **Availability:** A single host or site failure must not make the entire platform unavailable; redundant zones and tested recovery are needed.

- Original assumptions affected: The earlier host count, network capacity, storage growth, single-tenant access policies and cost estimate are no longer sufficient without revalidation.

- Web services and APIs become critical because the LMS, student systems, mobile applications and college portals need standardized, controlled communication. An API gateway can authenticate requests, apply rate limits, route traffic and provide logging.

- Multi-tenancy: Each college must have a unique tenant identity and logically separated data, roles, network policies and audit records. Tenant identifiers must be enforced by the application and database access layer, not merely supplied by the browser.

11.2 Capacity and Availability Evaluation (2 Marks)

Use the current peak CPU demand calculated in Task 3: 304 cores. Assume the new peak is five times this demand.

New peak CPU demand = 304 × 5 = 1,520 CPU cores

Host capacity = 32 physical cores; target utilization = 65%

Planned CPU demand per host = 32 × 0.65 = 20.8 cores

Hosts without failure reserve = 1,520 / 20.8 = 73.08

Round up = 74 hosts

With one-host failure reserve = 75 hosts

After one host fails: 74 × 32 × 0.65 = 1,539.2 cores

This is greater than 1,520 cores.

Memory estimate: Earlier assumed average memory demand was 768 GB. If peak memory is assumed to be 1.5 times average and then increases fivefold, estimated new peak memory = 768 × 1.5 × 5 = 5,760 GB. With 512 GB per host, 75 hosts provide 38,400 GB; after one host failure, 37,888 GB. Memory is therefore not the limiting factor under this simplified assumption. Actual memory use must be measured because workload growth may not scale linearly.

Storage and network: The question does not provide per-user storage or bandwidth figures, so an exact TB or Gbps value cannot be calculated honestly. Measure current peak storage growth, IOPS, throughput and concurrent sessions. For initial planning, reserve at least five times the measured peak workload capacity, plus separate backup/replication bandwidth and storage growth margin. Use object storage/CDN for large static learning content where suitable.

Availability: 99.9% annual availability allows approximately 0.1% downtime. 365 × 24 × 0.001 = 8.76 hours per year. This target requires redundant hosts, redundant network paths

and power, health checks, load balancing, automated recovery, multi-zone deployment, tested backups and clear maintenance procedures.

11.3 Adaptation or Redesign

Decision: MODIFY the original design into a hybrid, multi-zone, multi-tenant architecture. Retain the Type-1 virtualization foundation and shared storage principles, but add container orchestration, an API gateway, tenant-aware security, multi-zone deployment and cloud bursting for temporary peaks. A complete redesign is not necessary because the original virtualization base remains useful, but its capacity and availability design must be expanded.

11.4 Revised Service and Resource Policies

Use IaaS for controlled VM workloads and network foundations; use PaaS/managed database services where they reduce administration and support high availability; use containers and Kubernetes for scalable application services.
Use an API gateway for authentication, throttling, routing, versioning and audit logging.
Use separate namespaces, network policies, quotas and service identities for each tenant. For highly sensitive workloads, consider dedicated databases or dedicated node pools.
Use horizontal auto-scaling for stateless web/API services; scale databases carefully using tested read replicas, connection pooling and caching.
Set per-tenant CPU, memory, storage and request quotas to prevent one college from exhausting shared resources.
Replicate critical data across zones and maintain immutable backups in a separate region.

11.5 Evidence-Based Defence and Trade-offs

Area	Original design	Modified design
Peak CPU planning	16 hosts for current 304-core peak with N+1.	Approximately 75 hosts at five times peak under the same 32-core/65% assumptions, or use measured hybrid-cloud burst capacity.
Availability	Single cluster and secondary backup site proposed.	Multi-zone service deployment, redundant load balancers, automated

		recovery and secondary-region DR.
Tenant security	Basic VLAN and role separation.	Tenant-aware identity, data partitioning, namespace/network policies, quotas, per-tenant audit and optional dedicated resources.
Cost	Higher initial hardware cost but reduced annual OPEX estimate.	Higher variable cloud and managed-service costs during peaks; capacity reservations and autoscaling control waste.
Complexity	Conventional VM cluster administration.	More complex orchestration, API gateway, tenant governance and cloud operations.

Cost control strategy: Keep predictable baseline workloads on the university's private cluster; use auto-scaling and cloud bursting for short peak periods after performance and security testing. Apply budgets, quotas, tagging and cost alerts per college. Compare reserved capacity with on-demand pricing and avoid running peak capacity continuously when demand is seasonal.

Trade-offs: Multi-zone deployment improves resilience but increases infrastructure and network costs. Managed PaaS reduces operational effort but may increase vendor lock-in. Kubernetes supports scale and portability but requires trained administrators. Strong tenant isolation improves security but can reduce resource sharing efficiency. The university should balance these factors using measured load tests and a pilot with selected colleges.

Final decision: Modify the original design. Retain virtualization for the stable baseline, add container orchestration and API management for elastic digital learning services, and use multi-zone/hybrid-cloud capacity to handle five-times peak demand. The calculated 75-host figure is a theoretical all-on-premises peak plan; a hybrid approach can reduce capital expenditure if burst capacity is contractually available and tested. The 99.9% target must be verified through monitoring and failover exercises, not assumed from architecture alone.

REFERENCES

1. CSA15 – Cloud Computing and Big Data Analytics, Assignment 1 – CO1, question paper supplied by the faculty.

2. NIST, The NIST Definition of Cloud Computing, Special Publication 800-145.

3. Docker Documentation, https://docs.docker.com/

4. Kubernetes Documentation, https://kubernetes.io/docs/

5. VMware vSphere Documentation, https://docs.vmware.com/

6. Microsoft Learn – Hyper-V virtualization documentation, https://learn.microsoft.com/

7. Prometheus Documentation, https://prometheus.io/docs/

8. OWASP, Application Security Verification Standard and API Security guidance, https://owasp.org/